8/10/2026

Practical - 12

End - End Communication at Transport Layer

a) Implement echo clientserver using TCP/UDP sockets

Aim:

To implement echo clientserver using TCP/UDP sockets.

Program:

TCP:

```
import socket
import threading
#---Server---
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen()
    print("[SERVER] Listening on 127.0.0.1:5000")
    conn, addr = server.accept()
    print(f"[SERVER] Connected by {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit':
            print("[SERVER] Connection closed by client.")
            break
        print(f"[SERVER] Received: {data}")
        conn.send(f"Server received: {data}".encode())
    conn.close()
    server.close()
```

```
# --- Client ---
def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] Connected to server")

    while True:
        msg = input("Enter message (or 'quit' to exit): ")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}")
    client.close()
# --- MAIN ---
if __name__ == "__main__":
    server_thread = threading.Thread(
        target = start_server,
        daemon = True
    )
    server_thread.start()
    start_client()
```

Output:

```
C:\Users\Vedhasree S\Desktop\12 13 14 Experiments>python echo_clientserver_using_tcp.py
[SERVER] Listening on 127.0.0.1:5000
[CLIENT] Connected to server
[SERVER] Connected by ('127.0.0.1', 55081)
Enter message (or 'quit' to exit): hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
Enter message (or 'quit' to exit): hi
[SERVER] Received: hi
[SERVER RESPONSE] Server received: hi
Enter message (or 'quit' to exit): quit
[SERVER] Connection closed by client.
```

Program:

UDP:

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    while True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] Client {addr} ended the connection.")
            break
        print(f"[SERVER] Received from {addr}: {msg}")
        s.sendto(data, addr)
    s.close()
    print("[SERVER] Stopped.")
def udp_client():
    c = socket.socket(socket.AF_INET, socket.DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or 'quit' to stop): ")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] Closing connection...")
            break
        data, _ = c.recvfrom(1024)
        print("Echo from Server: ", data.decode())
```

```
    c.close()
    print("[CLIENT] Stopped.")

if __name__ == "__main__":
    server_thread = threading.Thread(
        target = udp_server,
        daemon = True
    )
    server_thread.start()
    udp_client()
```

Output:

```
C:\Users\Vedhasree S\Desktop\12 13 14 Experiments>python echo_clientserver_using_udp.py
[SERVER] Running on 127.0.0.1:12000
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 64639): hello
Echo from Server: hello
Enter message (or 'quit' to stop): hi
[SERVER] Received from ('127.0.0.1', 64639): hi
Echo from Server: hi
Enter message (or 'quit' to stop): quit
[SERVER] Client ('127.0.0.1', 64639) ended the connection.
[CLIENT] Closing connection...
[SERVER] Stopped.
[CLIENT] Stopped.
```

Result:
Thus, TCP/UDP echo client-server communication was successfully implemented using socket programming

9/10/2026

Practical 12

b) Implement a chat program using socket programming

Aim:
To implement chat program using socket programming

Program:
Server:

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('127.0.0.1', 5001))
server.listen(1)
print("Waiting for client...")
conn, addr = server.accept()
print("Client connected.")
while True:
    msg = conn.recv(1024).decode()
    if msg.lower() == "quit":
        break
    print("Client:", msg)
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "quit":
        break
conn.close()
server.close()
```

Client:

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('127.0.0.1', 5001))
print("Connected to server.")
```

```
while True:
    msg = input("Client: ")
    client.send(msg.encode())
    if msg.lower() == "quit":
        break
    reply = client.recv(1024).decode()
    print("Server: ", reply)
    if reply.lower() == "quit":
        break
client.close()
```

Output:

```
C:\Users\Vedhasree S\Desktop\12 13 14 Experiments>python chat_program
_server.py
Waiting for client...
Client connected.
Client: Hello
Server: Hi
```

```
C:\Users\Vedhasree S\Desktop\12 13 14 Experiments>python chat_program
_client.py
Connected to server.
Client: Hello
Server: Hi
Client: quit
```

Result:

Thus, a chat program was successfully implemented using socket programming for communication between the client and server.